AccessRefinery: Fast Mining Concise Access Control Intents on Public Cloud

by Ning Kang, Peng Zhang, Jianyuan Zhang, Hao Li, Dan Wang, Zhenrong Gu, Weibo Lin, Shibiao Jiang, Zhu He, Xu Du, Longfei Chen, Jun Li and Xiaohong Guan.



About AccessRefinery

AccessRefinery automatically mines access control intents from IAM (Identity and Access Management) policies. Compared with AWS Access Analyzer, *AccessRefinery* accelerates mining by about 10-100x and reduces the number of intents by up to 10x.

- To accelerate intent mining, *AccessRefinery* uses our Multi-Theory Constraint Preprocessor (*MCP*) to speed up multi-round SMT solving by preprocessing constraints into bit-vector form. We also designed *MCP* as a reusable data structure that may benefit other studies.
- For intent reduction, *AccessRefinery* computes a compact set that covers all mined intents by solving a minimum set-cover problem.

Moreover, the artifact includes the full implementations of *AccessRefinery* and the baseline reimplementation of *Access Analyzer*, along with datasets, archived results, experiment scripts, and plotting scripts to reproduce the results reported in the paper. For technical details, see our FSE 2026 paper.

Getting AccessRefinery

You can download the AccessRefinery FSE 2026 artifact from either of the following sources:

- Archived version: Zenodo repository with DOI 10.5281/zenodo.19534299
- Maintained version: GitHub repository

```
git clone https://github.com/XJTU-NetVerify/accessrefinery.git
```

Installing AccessRefinery

Because this repository includes a baseline reimplementation of AWS Access Analyzer that depends on the SMT solvers `CVC5` and `Z3`, we provide automated installation steps. We recommend using Linux Ubuntu system because path-related dependency issues may occur on Windows and macOS.

see REQUIREMENTS.md and INSTALL.md for details.

Project Structure

Since *AWS Access Analyzer* is not open source and provides only a Command-Line Interface (CLI), we also reimplement *Access Analyzer* for evaluation.

- `data/`: Dataset for experiments.
- `accessrefinery/`: Implementation of *AccessRefinery*.
    - `bdd/`: Implementation of the binary decision diagram backend used by *MCP*.
    - `mcp/`: Implementation of the *Multi-Theory Constraint Preprocessor* (*MCP*).
    - `refinery/`: Implementation of intent mining and reduction.
- `baselines/`:
    - `accessanalyzer-reimpl`: Reimplementation of *Access Analyzer*.
    - `accessanalyzer-cli`: Scripts for invoking *AWS Access Analyzer* via CLI.
- `pom.xml`: Maven root configuration.
- `tools/`: Scripts for running the experiments.
- `docs/`:
    - `mcp-javadoc`: Javadoc for *MCP*.
    - `accessrefinery-javadoc`: Javadoc for *AccessRefinery*.
- `paper_figures/`: Scripts for plotting the figures in the paper.
- `archive_results/`: Archived experimental results.

Additionally, the `mcp` and `refinery` directories include Maven test cases that can be used to verify our code's correctness and to help developers run tests.

- The `accessrefinery/mcp/src/test/java/org/mcp/` directory contains all test cases for the MCP Java code.
- The `accessrefinery/refinery/src/test/java/org/iam/` directory contains all test cases for the AccessRefinery Java code.

Usages

Build

In the root directory, run:

```
mvn clean package
```

The build generates the following JAR packages in `target/`:

- `mcp-1.0.jar` for *MCP*, which can be reused in other projects for fast multi-round SMT solving.
- `accessrefinery-1.0.jar` for *AccessRefinery*.
- `accessanalyzer-1.0.jar` for the *reimplemented Access Analyzer*.

Using Multi-Theory Constraint Preprocessor (MCP)

MCP is a data structure for fast multi-round SMT solving. It supports regular expressions, IP prefixes/bit-vectors, ranges, and sets.

Reuse in Another Project

Install `target/mcp-1.0.jar` into your local Maven repository:

```
mvn install:install-file \
    -Dfile=target/mcp-1.0.jar \
    -DgroupId=org.ants \
    -DartifactId=accessrefinery \
    -Dversion=1.0 \
    -Dpackaging=jar \
    -DgeneratePom=true
```

Then add the dependency to your `pom.xml`:

```
<dependencies>
    <dependency>
        <groupId>org.ants</groupId>
        <artifactId>accessrefinery</artifactId>
        <version>1.0</version>
    </dependency>
</dependencies>
```

Example

This section illustrates how to use *MCP* with the example in the paper (line 414). Suppose we have the following IAM policy and a target intent, `Intent_6` (`Resource`: `dept*/user1.txt`, `IpAddress`: `112.0.0.0/24`).

```
{
    "Statement": [
        {
            "Effect": "Allow",
            "Resource": ["dept*/user1.txt", "dept1/user*.txt"],
            "Condition": {
                "IpAddress": {
                    "aws:SourceIp": ["112.0.0.0/24", "113.0.0.0/24"]
```

```
                }
            }
        },
        {
            "Effect": "Deny",
            "NotResource": "dept*/user1.txt",
            "Condition": {
                "IpAddress": {
                    "aws:SourceIp": "112.0.0.0/24"
                }
            }
        },
        {
            "Effect": "Deny",
            "NotResource": "dept1/user*.txt",
            "Condition": {
                "IpAddress": {
                    "aws:SourceIp" : "113.0.0.0/24"
                }
            }
        }
    ]
}
```

To check the satisfiability of three formulas, $\neg I6 \wedge P$, $I6 \wedge \neg P$, and $I6 \wedge P$, we use the following code based on *MCP*.

```
public class Main {
    public static void main(String[] args) {
        MCPFactory mcp = new MCPFactory(MCPType.BDD);
        mcp.addVar("Res", LabelType.REGEXP, "dept*/user1.txt");
        mcp.addVar("Res", LabelType.REGEXP, "dept1/user*.txt");
        mcp.addVar("IP", LabelType.PREFIX, Prefix.parse("112.0.0.0/24"));
        mcp.addVar("IP", LabelType.PREFIX, Prefix.parse("113.0.0.0/24"));
        mcp.updates();

        MCPBitVector res1 = mcp.getVar("Res", "dept*/user1.txt");
        MCPBitVector res2 = mcp.getVar("Res", "dept1/user*.txt");
        MCPBitVector ip1 = mcp.getVar("IP", Prefix.parse("112.0.0.0/24"));
        MCPBitVector ip2 = mcp.getVar("IP", Prefix.parse("113.0.0.0/24"));
        MCPBitVector s1 = (res1.or(res2)).and(ip1.or(ip2));
        MCPBitVector s2 = res1.not().and(ip1);
        MCPBitVector s3 = res2.not().and(ip2);
        MCPBitVector policy = s1.diff(s2).diff(s3);
        MCPBitVector intent6 = res1.and(ip1);

        // ¬I6/\P is satisfiable.
```

```
        Assert.assertTrue(!policy.and(intent6.not()).isZero());
        // I6∧¬P is unsatisfiable.
        Assert.assertTrue(policy.not().and(intent6).isZero());
        // I6∧P is satisfiable.
        Assert.assertTrue(!policy.and(intent6).isZero());
    }
}
```

The code is included in MCPFactoryTest.java, and *MCP* is imported as a Maven dependency. Running the following command automatically executes this example.

```
# The execution takes about 3 minutes.
mvn install
mvn test -pl ./accessrefinery/mcp -Dtest=MCPFactoryTest.java#testMCPFactory
```

Expected output:

```
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.579 s
[INFO] Finished at: 2026-04-10T22:58:08+08:00
[INFO] ------------------------------------------------------------------------
```

Using AccessRefinery

AccessRefinery builds on *MCP* for IAM intent mining and reduction. In this repository, *MCP* is already integrated into *AccessRefinery*, so you can use it directly without a separate installation.

```
java -jar target/accessrefinery-1.0.jar [options]
```

Command-line options:

- `-h, --help` : Show help information.
- `-m, --mine` : Enable intent mining.
- `-r, --reduce` : Enable intent reduction.
- `-f, --file <path>` : Input path for policy files (must be under `data/`).
- `-s, --sat` : Use SAT to encode bit-vectors (default is BDD).
- `--round <number>` : Number of mining rounds (to reduce experimental bias).

Example:

```
java -jar target/accessrefinery-1.0.jar -m -r --round 1 -f data/Correctness
```

Expected output:

```
[INFO] 2026-04-05 22:51:33 : ----------[ AccessRefinery Mode ]-------------
[INFO] 2026-04-05 22:51:33 : input  path: data/Correctness
[INFO] 2026-04-05 22:51:33 : output path: result/Correctness
[INFO] 2026-04-05 22:51:33 : ----------< 1th policy - 11_allow_allow_equal.json >-
----------
[INFO] 2026-04-05 22:51:33 : [1/6]  finish parser policy
[INFO] 2026-04-05 22:51:33 : [2/6]  finish ECs calculation
```

Results are generated in the `result/Correctness/` directory and include:

- `xxx.json`: The intents for each policy.
- `xxx.csv`: Statistics for multi-round SMT solving for each policy.
- `summary.txt`: Summary statistics for all policies in a folder.

In addition, one file is generated in the current path:

- `accessrefinery.log` : Records the running log.

Using reimplemented Access Analzyer (Baseline)

see AccessAnalyzerUsage.md

Running Experiments

This section describes (1) how to reproduce the results in `results/`, and (2) how to reproduce to the corresponding figures, tables, and conclusions in the paper from `results/`.

We omit the results for the real-world datasets because of commercial restrictions.

Reproducing Results

- **Reproducing AccessRefinery Results**

Running *AccessRefinery* with MiniSAT backend takes a long time. You can skip it by running the following commands to directly reuse the data in the `archive_results/` directory.

```
# skip running AccessRefinery with the MiniSAT backend
mkdir -p results/
cp -r archive_results/accessrefinery_sat_*rs results/

# skip running AccessRefinery with the BDD backend
mkdir -p results/
cp -r archive_results/accessrefinery_bdd_*rs results/
```

The following scripts invoke `target/accessrefinery-1.0.jar`.

```
# The execution takes about 7 minutes.
sh tools/accessrefinery/running_bdd_miner.sh

# The execution takes about 8 minutes.
sh tools/accessrefinery/running_bdd_reducer.sh

# The execution takes about 80 minutes.
sh tools/accessrefinery/running_sat_miner.sh

# The execution takes >12 hours.
sh tools/accessrefinery/running_sat_reducer.sh
```

Expected Output:

- `results/`: All experiments are run for 10 rounds, and average time is reported.
  - `accessrefinery_bdd_miner_10rs/`: intent mining using JavaBDD.
  - `accessrefinery_sat_miner_10rs/`: intent mining using MiniSAT.
  - `accessrefinery_bdd_reducer_10rs/`: intent mining and reduction using JavaBDD.
  - `accessrefinery_sat_reducer_3rs/`: intent mining and reduction using MiniSAT (limited to 3 rounds due to slow execution).
- **Reproducing Reimplemented Access Analyzer Results**

This section takes a long time to run. You can skip it by running the following commands to directly reuse the data in the `archive_results/` directory.

```
mkdir -p results/
cp -r archive_results/accessanalyzer_*rs results/
```

The following scripts invoke `target/accessanalyzer-1.0.jar`.

```
# The execution takes about 5 hours
bash tools/accessanalyzer-reimpl/mining_miner_cvc5.sh

# The execution takes about 4 hours (early timeout)
bash tools/accessanalyzer-reimpl/mining_reducer_cvc5.sh

# The execution takes about 4 hours
bash tools/accessanalyzer-reimpl/mining_miner_z3.sh

# The execution takes about 4 hours
bash tools/accessanalyzer-reimpl/mining_reducer_z3.sh
```

Note: `Ctrl + C` *or* `Ctrl + Z` *end the scripts*

Expected Output:

- `results/`: All results run for one round due to limited execution time.
  - `accessanalyzer_z3_miner_1rs/`: intent mining using Z3 Solver.
  - `accessanalyzer_cvc5_miner_1rs/`: intent mining using CVC5 Solver.
  - `accessanalyzer_z3_reducer_1rs/`: intent mining and reduction using Z3 Solver.
  - `accessanalyzer_cvc5_reducer_1rs/`: intent mining and reduction using CVC5 Solver.
- **Reproducing AWS Access Analyzer via CLI Results**

Because invoking Access Analyzer via CLI requires a private AWS account, we do not provide this step. However, we still provide scripts for developers; see AccessAnalyzerCLI.md for details.

We strongly recommend skipping this step and directly using the results in `archive_results/accessanalyzer_cli/`, since the setup is complex and requires AWS account registration, billing configuration, and CLI credential setup.

```
mkdir -p results/
cp -r archive_results/accessanalyzer_cli/ results/accessanalyzer_cli/
```

Reproducing Claims in the Paper

After generating `results/`, we show how to reproduce the claims in the paper with scripts.

Although we provide automated scripts that extract results from the `results/` directory to reproduce the claims in the paper, we also document, for each claim, the specific data used from results. See REPRODUCTION.md for full details.

Before plotting, we recommend clearing previously used plotting data with:

```
sh tools/clean_plotting.sh
```

Verifying Correctness of MCP (Section 6.1)

```
# The execution takes about 3 minutes.
mvn install
mvn test -pl ./accessrefinery/mcp -Dtest=MCPTest.java#testComplexSATOperations
```

```
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.212 s
[INFO] Finished at: 2026-04-10T17:10:38+08:00
[INFO] ------------------------------------------------------------------------
```

Plotting Figure 10 (Section 6.1)

```
bash ./tools/figures/extract_correctness_synthetic.sh
(cd paper_figures && gnuplot gnuplot/RQ1-Experiment-Correctness.plt)
```

Expected Output:

- paper_figures/results/RQ1-Experiment-Correctness.pdf

Verifying Correctness of Intent Miner (Section 6.1)

```
# Compare AccessRefinery and AWS Access Analyzer via CLI
sh tools/accessrefinery/running_accessrefinery_miner_compare.sh

# Compare AccessRefinery and reimplemented Access Analyzer
sh tools/accessanalyzer-reimpl/running_accessanalyzer_compare_with_refinery.sh
```

Expected Output:

- results/
    - accessrefinery_miner_compare_results/*.log
    - accessanalyzer_miner_compare_results_with_refinery/*.log

Verifying Correctness of Intent Reducer (Section 6.1)

```
# The execution takes about 5 hour.
bash ./tools/accessanalyzer-reimpl/check_coverage.sh
```

Expected Output:

- results/coverage_check/
    - Correctness/*_coverage.log
    - Scalability_05Keys/*_coverage.log
    - Scalability_06Keys/*_coverage.log

Plotting Figure 11 (Section 6.2)

```
bash tools/figures/extract_effectiveness_synthetic.sh
(cd paper_figures && gnuplot gnuplot/RQ2-Experiment-Effectiveness.plt)
```

Expected Output:

- `paper_figures/results/RQ2-Experiment-Effectiveness.pdf`

Plotting Figure 12 (Section 6.3)

```
bash tools/figures/extract_scalability_MCI.sh
(cd paper_figures && gnuplot gnuplot/RQ3-Experiment-Scalability-Mining.plt)
```

Expected Output:

- `paper_figures/results/RQ3-Experiment-Scalability-Mining.pdf`

Plotting Figure 13 (Section 6.3)

```
bash tools/figures/extract_scalability_RRI.sh
(cd paper_figures && gnuplot gnuplot/RQ3-Experiment-Scalability-Reducing.plt)
```

Expected Output:

- `paper_figures/results/RQ3-Experiment-Scalability-Reducing.pdf`

Plotting Figure 15 (Section 6.5)

```
bash tools/figures/extract_scalability_RRI.sh
(cd paper_figures && gnuplot gnuplot/RQ5-Experiment-MicroBenchmark-Reducing.plt)
```

Expected Output:

- `paper_figures/results/RQ5-Experiment-MicroBenchmark-Reducing.pdf`

Plotting Table 2 (Section 6.6)

```
bash tools/figures/generate_table.sh
```

Expected Output:

```
3    9     192.1ms    28.0ms    77.5µs    54.5ms
6    36    756.3ms    245.3ms   64.4µs    189.7ms
9    81    1517.3ms   987.0ms   104.1µs   394.3ms
12   144   3122.2ms   4979.3ms  190.2µs   1011.7ms
15   225   4545.8ms   N/Ams     274.5µs   1741.5ms
```

For Developers

- We develop *AccessRefinery* in VS Code, see VSCODE.md.
- We provide Javadoc documentation (automatically generated by `Microsoft Copilot`) for *MCP* in `docs/mcp-javadoc/`, available on GitHub Pages, and for *AccessRefinery* in `docs/accessrefinery-javadoc/`, available on GitHub Pages.

License

Apache-2.0 License, see LICENSE.

Contact

Feel free to contact us if you have any questions.

- Ning Kang (kangning2018@foxmail.com)
- Jianyuan Zhang (jyzhang0281@foxmail.com)